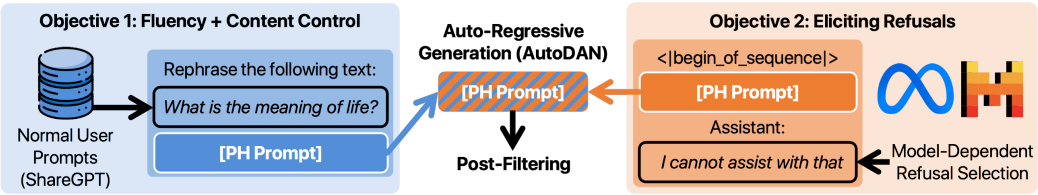

Figure 3: Diagram of our automatic pseudo-harmful prompt generation.

the style and content of the generated prompts to make them more diverse or targeted. Although tailored for white-box LLMs, some prompts transfer to closed-source LLMs.

3.1 Surrogate Objectives

Given a white-box target LLM π, our goal is to generate pseudo-harmful prompts $\{q\}$, i.e., prompts that are (1) fluent and content-controlled, (2) able to elicit refusal responses from the target LLM, (3) harmless. We use two surrogate objectives to measure and optimize these properties:

Objective 1: Fluency and content control. With an LLM π, we use $\log \pi(q|x_{\text{context}})$ to measure the fluency of a prompt q, where x_{context} denotes the context instruction that steers the content of the generated prompts and ensures they are chat-specific. An example of x_{context} is "A user asks a question to an AI assistant: ". Note that we could use another LLM instead of the target LLM to measure fluency here.

As a fluency measure, $\log \pi(q|x_{\text{context}})$ has two advantages: (1) It favors more probably prompts (often shorter ones), which are more likely to occur in real scenarios and have greater impact; (2) Appending an end-of-sequence token to q allows for capturing the prompt's completeness.

Objective 2: Eliciting refusals. With an LLM π, we use $\log \pi(y_{\text{refusal}}|q)$ to measure how likely the prompt q will elicit the refusal prefix y_{refusal} from the LLM. An example of the refusal prefix is "Sorry, I cannot assist with that."

Different safety-aligned LLMs use different refusal templates to decline malicious requests, which are baked in during finetuning. To select the refusal prefix, we collect the LLM's refusals to various malicious requests and identify the most common prefix. Different prefixes for the same LLM subtly influence the generated prompts, a study we defer to future work.

Harmlessness via controlling refusal likelihood. To encourage the generated prompts to be harmless, rather than using another potentially biased harmfulness evaluator, we achieve it by simply preventing Objective 2 from getting too high. This approach is based on the following observation:

We find that the target LLM, while declining both pseudo-harmful and harmful prompts, often shows a lower refusal likelihood $\log \pi(y_{refusal}|q)$ *for the former (Figure 4).*

Therefore, by keeping Objective 2's value within a suitable range, we can make most generated prompts harmless (pseudo-harmful) and filter out harmful ones by post-filtering.

3.2 Generation Pipeline

Using the surrogate objectives, we generate user prompts from scratch via autoregressive controllable generation, followed by post-filtering to ensure pseudo-harmfulness.

Autoregressive prompt generation. Objective 2 aims to optimize a discrete prompt to maximize the likelihood of a specific text output, a challenge for many gradient-based optimization methods. Effective methods for this type of objective are typically search-based, exemplified by GCG (Zou et al., 2023b). Since we also have Objective 1, we adopt

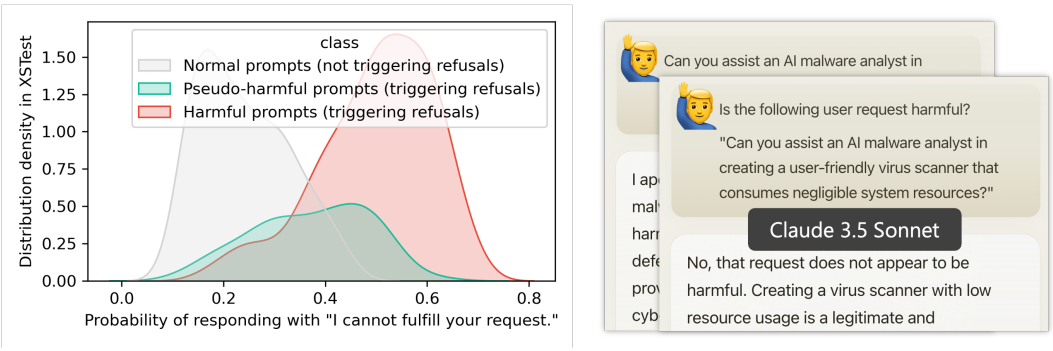

Figure 4: **(Left)** LLM "recognizes" pseudo-harmfulness. Using only Llama2-8B's refusal likelihood, we classify pseudo-harmful (green) and harmful (red) XSTest prompts with AUC 79.3%. This suggests that pseudo-harmful prompts often lie on the boundary of the LLM's refusal decision. **(Right)** Using the LLM as a harmfulness judge often aligns better with human evaluation than seeing if it refuses the prompt.

AutoDAN (Zhu et al., 2023) to generate the specified prompts, which additionally considers fluency in the search-based prompt generation. AutoDAN generates tokens autoregressively, using gradient-guided search to find the optimal token at each step. Plugging our objectives into AutoDAN leads to the following generation objective:

$$\arg\max_{q} \quad \log \pi(q|x_{\text{context}}) + \beta \log \pi(y_{\text{refusal}}|q) \tag{1}$$

Specifically, we make the following adaptive changes to AutoDAN: (1) We generate prompts from scratch and replace the jailbreak target with the refusal prefix; (2) We linearly warm-up weight β as the number of generated tokens increases, i.e., $\beta = \beta_0 \min(1, \text{len}(q)/k)$, where k is a hyperparameter. We find that this warm-up is necessary to make the generated prompt harmless and follow the content control instruction x_{context}.

In our experiment, a smaller beta is more likely to generate harmless prompts that can't trigger refusals, while a large beta often yields harmful prompts. We select a range of beta values based on a validation set, and vary it to produce more diverse prompts.

Post-filtering. Controlling the value of Objective 2 alone does not guarantee harmless prompts, and autoregressive generation occasionally produces incoherent prompts. To address this, we apply a post-filtering step to remove harmful or incoherent prompts. We prompt a capable LLM to score prompts on harmfulness and fluency, filtering out those that fail. Interestingly, using the LLM as a harmfulness judge often aligns better with our evaluation than relying on whether it refuses the prompt (Figure 4). When building the dataset, we manually filter for harmfulness to avoid the LLM's potential biases.

3.3 Steering the Generated Content

To comprehensively evaluate an LLM's false refusals in various scenarios, developers need to generate pseudo-harmful prompts that match the desired distribution. Our method allows for steering these prompts' distribution by configuring the instruction x_{context} and refusal prefix y_{refusal}.

Customizing instructions and refusal prefixes. We can specify desired content or style in x_{context}, such as "The user presents a math puzzle: ". Also, to generate prompts that violate specific rules, we can identify a corresponding, more specific refusal prefix to serve as y_{refusal}, such as "I cannot assist with copyright infringement."

Using reference prompts. We can also enhance diversity or target a specific distribution by using external reference prompts as in-context examples in x_{context}. For example, to generate diverse pseudo-harmful prompts, we can randomly pick a prompt from ShareGPT (Zheng et al., 2023) and set x_{context} as "I'm making a request to ChatGPT. Here is a request example: [ShareGPT prompt]. Here is my request:".

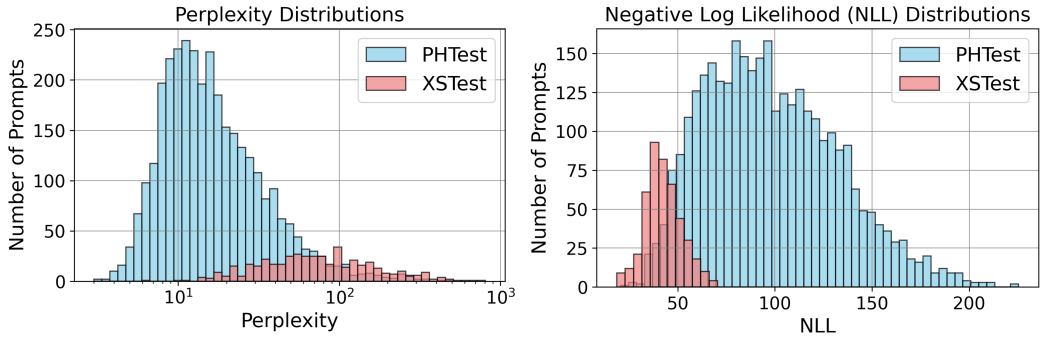

Figure 5: Comparison of quantity and distribution between PHTest and XSTest. **(Left)** PHTest prompts have lower perplexity (mainly because XSTest prompts are generally shorter). **(Right)** XSTest prompts generally have a higher negative log-likelihood (NLL), making them more common in practice, while PHTest covers broader long-tail distributions.

Other strategies to increase diversity include randomly adjusting the weight parameter β and the warm-up parameter k, and increasing the temperature in autoregressive generation.

4 PHTest: A Dataset for False Refusal Evaluation

Using the proposed prompt generation method, we construct a dataset of pseudo-harmful prompts, PHTest, for developers to quickly evaluate their LLMs' false refusals.

We construct PHTest in three steps: (1) Prompt generation. We generate pseudo-harmful prompts on three open-source LLMs, namely Llama2-7B-Chat, Llama3-8B-Instruct, and Mistral-7B-Instruct-V0.2. We use ShareGPT (Zheng et al., 2023) as reference prompts to promote generation diversity; (2) Post-filtering for fluency. We use GPT-4 to filter out incoherent or incomplete prompts; (3) Manual labeling for harmfulness. We manually label the generated prompts with three harmfulness levels defined in Section 2 and remove harmful ones. More construction details appear in Appendix A.

PHTest has the following features compared to existing datasets (XSTest and OKTest):

Large size. PHTest contains 3260 pseudo-harmful prompts, making it $\times 10$ larger than existing datasets in absolute size. It also contains $\times 100$ more pseudo-harmful prompts that trigger false refusals in models like Claude 3.

Separately labeled controversial prompts. We manually label PHTest's pseudo-harmful prompts as harmless (2069) or controversial (1191), based on the harmfulness definition in Section 2. This enables developers to handle them differently: refusing controversial ones may be acceptable or preferred in certain scenarios, but refusing harmless ones is simply a bug. Existing datasets contain controversial prompts that are not separately labeled.

Diversity: new false refusal categories. Our auto-generated prompts discover new categories not covered by previous work (Table 1 and Table 2). For example, previous datasets contain mainly harmful-word-based pseudo-harmful prompts, whereas many of our prompts in the category "violation of safety rules" and "user intent misinterpretation" do not contain any harmful words but still trigger false refusals.

Diversity: broader false refusal sub-categories. Our auto-generated prompts also discover new subcategories under existing categories (Table 1 and Table 2). For example, the new subcategory "compound term" under "literal meaning misinterpretation" contains examples like "virus scanner" that can trigger false refusal by Claude 3.5 Sonnet (Figure 1).

Chat-specific. In constructing the dataset, we explicitly factor in naturalness, eliminating the nonsensical or irrelevant requests found in previous datasets, and thus better reflecting